

Multiple enemies implementation for Maze Chase

Model: claude-sonnet-5-5

User

30/09/2026, 16:12:01

Task 1 — Multiple Enemies

I have uploaded the complete original Maze Chase project as a ZIP.

If you can read the files inside the ZIP only then do the below work, if not then tell me and I will share you individual files. Do not Hallucinate.

Read README.md and inspect the existing code. Implement only Task 1: Multiple Enemies exactly according to the README.

Requirements:

- Add 2 more enemies, for 3 enemies total.
- Start them at different valid corners/positions.
- Each enemy must independently chase the player using the existing BFS pathfinding.
- Use a `self.enemies` list and update, draw, and collision-check every enemy.
- Restarting must correctly reset all enemies.

Important:

- Preserve all pre-existing code and behavior unless a change is strictly necessary to implement Task 1.
- Do not rewrite, refactor, optimize, or fix unrelated existing code.
- Do not implement Tasks 2, 3, or 4.

- Do not add unrelated features, files, or dependencies.
- Keep changes minimal and consistent with the existing code style.

Return:

1. The complete updated contents only for files that need modification, clearly labeled with their file paths.
2. Put all code in directly copy-pastable code blocks so I can paste it into the corresponding existing files in VS Code.
3. Do not use diff/patch format or partial snippets. Give the complete contents of each modified file.
4. Give a very short summary of what changed.
5. Give a short manual test checklist for Task 1.

File: PES1UG24CS316_LAB4_11_maze_chase.zip ↕

Assistant

30/09/2026, 16:12:56

► Unzip and list project files `bash_tool`

The ZIP opens fine. It also bundles a `.venv` and `.git`, so I'll list only the project's own files.

► List project files and read README `bash_tool`

► Read all source files `bash_tool`

► Setting up a grid game with enemies and corner placements.

Only `game/game_engine.py` references the enemy, so that's the only file that needs to change. I'll make the edit and test it headless.

► Apply Task 1 edit and diff against original `bash_tool`

► Diff original vs modified, install pygame `bash_tool`

The edit applied cleanly. Next is a headless test of the behaviour: three enemies, independent BFS chasing, collision with any enemy, and restart.

► Headless functional test of Task 1 `bash_tool`

► Rethinking a flawed distance test for maze pathfinding.

► Switching test to measure actual BFS path length

► Corrected headless test using BFS path length `bash_tool`

► Print final file for the answer `bash_tool`

I could read the ZIP and the full project. Only one file needs to change for Task 1.

`game/game_engine.py`

```
import pygame
from game.maze import generate_maze, CELL
from game.entities import Player, Enemy

COLS, ROWS = 13, 11
WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 50
FPS = 60

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Chase")
        self.clock = pygame.time.Clock()
```

```
self.font = pygame.font.SysFont("monospace", 22)
self.big_font = pygame.font.SysFont("monospace", 38, bold=True)
self.reset()

def reset(self):
    self.walls = generate_maze(COLS, ROWS)
    self.player = Player(0, 0)
    self.enemies = [Enemy(ROWS-1, COLS-1), Enemy(0, COLS-1), Enemy(ROWS-1, 0)]
    self.exit_rect = pygame.Rect((COLS//2)*CELL+5, (ROWS//2)*CELL+5, CELL-10, CELL-10)
    self.caught = False
    self.won = False

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT: return False
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r: self.reset()
    return True

def update(self):
    if self.caught or self.won: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)
    for enemy in self.enemies:
        enemy.update(self.walls, self.player, ROWS, COLS)
        if self.player.rect.colliderect(enemy.rect):
            self.caught = True
    if self.player.rect.colliderect(self.exit_rect):
        self.won = True

def draw(self):
    self.screen.fill((230, 220, 210))
    wc=(50,40,60)
    for r in range(ROWS):
```

```

        for c in range(COLS):
            x,y=c*CELL,r*CELL
            w=self.walls[r][c]
            if w[0]: pygame.draw.line(self.screen,wc,(x,y),(x+CELL,y),3)
            if w[1]: pygame.draw.line(self.screen,wc,(x,y+CELL),(x+CELL,y+CELL),3)
            if w[2]: pygame.draw.line(self.screen,wc,(x+CELL,y),(x+CELL,y+CELL),3)
            if w[3]: pygame.draw.line(self.screen,wc,(x,y),(x,y+CELL),3)
pygame.draw.rect(self.screen,(80,200,80),self.exit_rect,border_radius=4)
lbl=self.font.render("EXIT",True,(20,80,20))
self.screen.blit(lbl,(self.exit_rect.x+2,self.exit_rect.y+6))
self.player.draw(self.screen)
for enemy in self.enemies:
    enemy.draw(self.screen)
hud=pygame.Rect(0,ROWS*CELL,WIDTH,50)
pygame.draw.rect(self.screen,(30,30,50),hud)
info=self.font.render("Reach EXIT before the enemy catches you!  R=Restart",True,(200,200,200))
self.screen.blit(info,(8,ROWS*CELL+14))
if self.caught:
    self._overlay("CAUGHT!", (220,60,60))
if self.won:
    self._overlay("ESCAPED!", (80,220,80))
pygame.display.flip()

def _overlay(self, text, color):
    surf=pygame.Surface((WIDTH,ROWS*CELL),pygame.SRCALPHA)
    surf.fill((0,0,0,140))
    self.screen.blit(surf,(0,0))
    msg=self.big_font.render(text,True,color)
    sub=self.font.render("Press R to Restart",True,(200,200,200))
    self.screen.blit(msg,(WIDTH//2-msg.get_width()//2,ROWS*CELL//2-30))
    self.screen.blit(sub,(WIDTH//2-sub.get_width()//2,ROWS*CELL//2+20))

def run(self):

```

```
running=True
while running:
    running=self.handle_events()
    self.update()
    self.draw()
    self.clock.tick(FPS)
pygame.quit()
```

Summary of changes

Three edits, all in `game_engine.py`:

- `reset()` now builds `self.enemies` with 3 enemies instead of one `self.enemy`. The original bottom-right enemy stays, and the new ones start at the top-right `(0, COLS-1)` and bottom-left `(ROWS-1, 0)`.
- `update()` loops over the enemies. Each one runs its own `update()`, which calls `bfs()`, and touching any enemy sets `caught`.
- `draw()` loops over the enemies.

`entities.py`, `maze.py`, `main.py` and the README are unchanged. Restart works because `reset()` rebuilds the whole list.

I ran the game headless and checked the following:

- There are 3 enemies at distinct corners.
- After one move, every enemy is exactly one step closer along its own BFS path.
- Being touched by the 2nd or 3rd enemy triggers CAUGHT.
- Pressing R restores all 3 enemies to their start corners.

I could not test the visuals or the real keyboard, so please run through the checklist below.

Manual test checklist

1. Run `python main.py`. You should see 3 red enemies, one in each of the bottom-right, top-right and bottom-left corners. The player is blue at top-left.
2. Stay still for a few seconds. All 3 enemies should move toward you independently, taking different routes.
3. Let each enemy reach you in separate runs. Every one of them should trigger the CAUGHT overlay.
4. Press R when caught, or in the middle of a run. A new maze should appear with all 3 enemies back at their start corners and the CAUGHT or ESCAPED overlay cleared.
5. Reach the EXIT before they catch you. ESCAPED should still show, and enemies should freeze afterwards as before.

One thing you might notice: the HUD text still says "the enemy" (singular). I left it alone because changing it isn't part of Task 1.

User

30/09/2026, 16:21:35

Task 2 — Speed Up Over Time

Task 1 (Multiple Enemies) has already been implemented and tested successfully.

Using the current project code from Task 1, implement only Task 2: Speed Up Over Time from README.md.

Requirements:

- Track elapsed game time using `pygame.time.get_ticks()`.
- Every 15 seconds, make the enemies move faster by reducing each enemy's `move_interval` by 2.
- `move_interval` must never go below 5.

- Apply the speed increase to all 3 enemies.
- Display the current enemy speed tier in the HUD.
- Restarting with R must reset the timer, enemy speeds, and speed tier.

Important:

- Preserve the existing Task 1 implementation and all pre-existing functionality.
- Change existing code only where strictly necessary for Task 2.
- Do not rewrite, refactor, optimize, or fix unrelated code.
- Do not implement Tasks 3 or 4.
- Do not add unrelated features, files, or dependencies.
- Keep changes minimal and consistent with the existing code style.

Return:

1. The complete updated contents only for files that need modification, clearly labeled with their file paths.
2. Put the code in directly copy-pastable code blocks.
3. Do not use diff/patch format or partial snippets. Give the complete contents of each modified file.
4. Give a very short summary of what changed.
5. Give a short manual test checklist specifically for Task 2.

Assistant

30/09/2026, 16:22:44

► Enlarging the HUD to fit a second display line.

Task 2 touches only `game/game_engine.py`. One design problem: the HUD bar is 50px tall and the existing line of text already fills it, so I need to check the text width before deciding where the speed-tier line can go.

► Measure HUD text width vs window width `bash_tool`

► Expanding the HUD bar to add a tier line

► Apply Task 2 edits and diff vs Task 1 version `bash_tool`

The diff shows only the intended changes. Now a headless test with a fake clock, covering timing, the floor at 5, all 3 enemies, and reset.

► Headless test of Task 2 with fake clock `bash_tool`

► Debugging why the enemy move counter stays at zero.

► Rerunning enemy-movement check with player at default start

► Confirm lower interval means more enemy moves `bash_tool`

► Print final file `bash_tool`

`game/game_engine.py`

```
import pygame
from game.maze import generate_maze, CELL
from game.entities import Player, Enemy

COLS, ROWS = 13, 11
WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 80
FPS = 60
SPEED_UP_MS = 15000 # enemies speed up every 15 seconds
SPEED_STEP = 2
MIN_MOVE_INTERVAL = 5

class GameEngine:
```

```
def __init__(self):
    pygame.init()
    self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Maze Chase")
    self.clock = pygame.time.Clock()
    self.font = pygame.font.SysFont("monospace", 22)
    self.big_font = pygame.font.SysFont("monospace", 38, bold=True)
    self.reset()

def reset(self):
    self.walls = generate_maze(COLS, ROWS)
    self.player = Player(0, 0)
    self.enemies = [Enemy(ROWS-1, COLS-1), Enemy(0, COLS-1), Enemy(ROWS-1, 0)]
    self.exit_rect = pygame.Rect((COLS//2)*CELL+5, (ROWS//2)*CELL+5, CELL-10, CELL-10)
    self.caught = False
    self.won = False
    self.start_ticks = pygame.time.get_ticks()
    self.speed_tier = 1

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT: return False
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r: self.reset()
    return True

def update(self):
    if self.caught or self.won: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)
    elapsed = pygame.time.get_ticks() - self.start_ticks
    target_tier = elapsed // SPEED_UP_MS + 1
    while self.speed_tier < target_tier and any(e.move_interval > MIN_MOVE_INTERVAL for e in self.enemies):
        self.speed_tier += 1
```

```

        for enemy in self.enemies:
            enemy.move_interval = max(MIN_MOVE_INTERVAL, enemy.move_interval - SPEED_STEP)
    for enemy in self.enemies:
        enemy.update(self.walls, self.player, ROWS, COLS)
        if self.player.rect.colliderect(enemy.rect):
            self.caught = True
    if self.player.rect.colliderect(self.exit_rect):
        self.won = True

def draw(self):
    self.screen.fill((230, 220, 210))
    wc=(50,40,60)
    for r in range(ROWS):
        for c in range(COLS):
            x,y=c*CELL,r*CELL
            w=self.walls[r][c]
            if w[0]: pygame.draw.line(self.screen,wc,(x,y),(x+CELL,y),3)
            if w[1]: pygame.draw.line(self.screen,wc,(x,y+CELL),(x+CELL,y+CELL),3)
            if w[2]: pygame.draw.line(self.screen,wc,(x+CELL,y),(x+CELL,y+CELL),3)
            if w[3]: pygame.draw.line(self.screen,wc,(x,y),(x,y+CELL),3)
    pygame.draw.rect(self.screen,(80,200,80),self.exit_rect,border_radius=4)
    lbl=self.font.render("EXIT",True,(20,80,20))
    self.screen.blit(lbl,(self.exit_rect.x+2,self.exit_rect.y+6))
    self.player.draw(self.screen)
    for enemy in self.enemies:
        enemy.draw(self.screen)
    hud=pygame.Rect(0,ROWS*CELL,WIDTH,80)
    pygame.draw.rect(self.screen,(30,30,50),hud)
    info=self.font.render("Reach EXIT before the enemy catches you! R=Restart",True,(200,200,200))
    self.screen.blit(info,(8,ROWS*CELL+14))
    maxed = all(e.move_interval <= MIN_MOVE_INTERVAL for e in self.enemies)
    tier=self.font.render(f"Enemy Speed: Tier {self.speed_tier}"+(" (MAX)" if maxed else ""),True,(255,200,80))
    self.screen.blit(tier,(8,ROWS*CELL+46))

```

```

    if self.caught:
        self._overlay("CAUGHT!", (220,60,60))
    if self.won:
        self._overlay("ESCAPED!", (80,220,80))
pygame.display.flip()

def _overlay(self, text, color):
    surf=pygame.Surface((WIDTH,ROWS*CELL),pygame.SRCALPHA)
    surf.fill((0,0,0,140))
    self.screen.blit(surf,(0,0))
    msg=self.big_font.render(text,True,color)
    sub=self.font.render("Press R to Restart",True,(200,200,200))
    self.screen.blit(msg,(WIDTH//2-msg.get_width()//2,ROWS*CELL//2-30))
    self.screen.blit(sub,(WIDTH//2-sub.get_width()//2,ROWS*CELL//2+20))

def run(self):
    running=True
    while running:
        running=self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
    pygame.quit()

```

Summary of changes

Only `game/game_engine.py` changed:

- Constants: added `SPEED_UP_MS`, `SPEED_STEP` and `MIN_MOVE_INTERVAL`.

- `reset()`: stores `start_ticks` from `pygame.time.get_ticks()` and sets `speed_tier = 1`. Enemies are rebuilt at `move_interval = 20`, so R resets the timer, the speeds and the tier.
- `update()`: works out the tier from elapsed time. Each new tier lowers every enemy's `move_interval` by 2, never below 5. Tiers stop at 9, when the interval hits 5.
- `draw()`: shows `Enemy Speed: Tier N`, with `(MAX)` once the floor is reached.
- HUD size: I made the HUD bar 30px taller (`HEIGHT` and the HUD rect, 50 to 80). The existing text line already fills the 50px bar, so the tier needed its own line. The original HUD text is untouched.

Tier 1 is the starting speed of 20 frames per move, and 15 seconds later the interval drops to 18. Tier 9 is the maximum, at interval 5 after about 2 minutes.

I tested this headless with a fake clock and it passed:

- Nothing changes at 14.999s, and the speed-up happens at 15s.
- The interval steps 20→18→...→6→5 and never goes below 5.
- The same value applies to all 3 enemies.
- A long stall catches up correctly.
- R resets the timer, the intervals and the tier.
- Task 1 collision still works.
- Enemies made 12 moves per second at interval 5, against 3 at interval 20.

I couldn't check the on-screen look or real-time behaviour, so please run the checklist.

Manual test checklist

1. Launch the game. The HUD should show `Enemy Speed: Tier 1` in yellow under the original text, and the window is slightly taller.

2. Stay alive for about 15 seconds. The HUD should change to Tier 2 and all 3 enemies should visibly move faster. Tier 3 should follow at about 30 seconds.
3. Keep surviving, for example by hiding far from the enemies. At about 2 minutes 15 seconds it should read `Tier 9 (MAX)` and stop increasing.
4. Press R after reaching a higher tier. The HUD should return to Tier 1, the enemies should move at the slower starting speed, and the next speed-up should come 15 seconds after the restart, not after launch.
5. Get CAUGHT or reach EXIT. The tier should freeze while the overlay is shown, and R should reset it.
6. Confirm the Task 1 behaviour still holds: 3 enemies, each one catches you, and R resets the positions.

To reach the MAX tier quickly while testing, temporarily set `SPEED_UP_MS = 2000` and restore it to `15000` afterwards.

User

30/09/2026, 16:30:48

Task 3 — Power Pellet (Freeze Enemy)

Tasks 1 and 2 have already been implemented and tested successfully.

Using the current project code, implement only Task 3: Power Pellet (Freeze Enemy) from README.md.

Requirements:

- Add a yellow circular power pellet at a valid location in the maze.
- When the player collects the pellet, freeze the enemies for 5 seconds (300 frames at 60 FPS), as specified in the README.
- While frozen, enemies must not move/chase the player.
- Show a clear frozen indicator while the freeze effect is active.

- The pellet should disappear after being collected.
- Apply the freeze behavior correctly to all 3 enemies from Task 1.
- After the freeze duration ends, all enemies must resume chasing normally.
- Restarting with R must reset the pellet and freeze state.
- Preserve the Task 2 speed progression.

Important:

- Preserve the existing implementations of Tasks 1 and 2 and all pre-existing functionality.
- Change existing code only where strictly necessary for Task 3.
- Do not rewrite, refactor, optimize, or fix unrelated code.
- Do not implement Task 4.
- Do not add unrelated features, files, or dependencies.
- Keep changes minimal and consistent with the existing code style.

Return:

1. The complete updated contents only for files that need modification, clearly labeled with their file paths.
2. Put the code in directly copy-pastable code blocks.
3. Do not use diff/patch format or partial snippets. Give the complete contents of each modified file.
4. Give a very short summary of what changed.
5. Give a short manual test checklist specifically for Task 3.

Assistant

30/09/2026, 16:32:46

► Writing the exact code edits for pellet and freeze features.

Task 3 needs changes in two files. `entities.py` gets the `frozen` flag, the early return in `update()` and the on-enemy indicator. `game_engine.py` gets the pellet, the countdown and the HUD text. Let me apply it.

► Apply Task 3 edits and diff vs Task 2 `bash_tool`

► Verifying pellet placement rules and testing headless mode.

► Confirming the diff, now running a headless walk-to-pellet test

► Headless functional test of Task 3 `bash_tool`

► Tracing the player's path through the grid puzzle.

One assertion failed. Before deciding whether it's a test problem or a game bug, I'll print the state at that point.

► Diagnose which assertion failed `bash_tool`

► Debugging a failing test assertion involving game state flags.

► Fixing test expectation and rerunning the full suite

► Rerun Task 3 tests with corrected expectation `bash_tool`

Every logic check passes. Next I'll look at the rendered frames to make sure the pellet and the frozen indicator are actually clear on screen.

► Render the un-frozen state with pellet `bash_tool`

► Check pellet appearance before collection `view`

► Reviewing screenshot details for pellet visibility and HUD text.

The pellet reads clearly as a yellow circle in an open cell, away from the start, the corners and the EXIT. Now the frozen state.

► Check frozen indicator on enemies and HUD `view`

The HUD text and the pellet disappearing both work. The enemies aren't visible only because my test parked them off-screen. I'll render a frame with them in view to check the on-enemy indicator.

► Zoom into frozen enemy rendering `bash_tool`

► Zoomed view of frozen enemies `view`

All three frozen enemies get a light-blue ice outline, which stands out clearly against the red. Now the final files.

► Print final versions of both modified files `bash_tool`

Two files changed for Task 3: `game/entities.py` and `game/game_engine.py`.

`game/entities.py`

```
import pygame
from game.maze import CELL

SPEED = 2

class Player:
    def __init__(self, r, c):
        self.r, self.c = r, c
        cx, cy = c*CELL+CELL//2, r*CELL+CELL//2
        self.rect = pygame.Rect(cx-10, cy-10, 20, 20)
        self.color = (60, 120, 220)

    def move(self, keys, walls, rows, cols):
        dx=dy=0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx=-SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx=SPEED
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy=-SPEED
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy=SPEED
```

```

        nr = self.rect.move(dx,0)
        if self._valid(nr, walls, rows, cols): self.rect=nr
        nr = self.rect.move(0,dy)
        if self._valid(nr, walls, rows, cols): self.rect=nr

    def _valid(self, rect, walls, rows, cols):
        for px,py in [(rect.left,rect.top),(rect.right-1,rect.top),(rect.left,rect.bottom-1),(rect.right-1,rect.bottom-1)]:
            cr,cc=py//CELL,px//CELL
            if not(0<=cr<rows and 0<=cc<cols): return False
        return True

    def draw(self, screen):
        pygame.draw.ellipse(screen, self.color, self.rect)

class Enemy:
    def __init__(self, r, c):
        self.r, self.c = r, c
        cx, cy = c*CELL+CELL//2, r*CELL+CELL//2
        self.rect = pygame.Rect(cx-12, cy-12, 24, 24)
        self.color = (220, 60, 60)
        self.timer = 0
        self.move_interval = 20 # frames between cell moves
        self.frozen = False

    def update(self, walls, player, rows, cols):
        from game.maze import bfs
        if self.frozen: return
        self.timer += 1
        if self.timer >= self.move_interval:
            self.timer = 0
            pr, pc = player.rect.centery//CELL, player.rect.centerx//CELL
            step = bfs(walls, (self.r, self.c), (pr, pc), rows, cols)
            if step:

```

```

        dr, dc = step
        self.r += dr; self.c += dc
        cx, cy = self.c*CELL+CELL//2, self.r*CELL+CELL//2
        self.rect.center = (cx, cy)

def draw(self, screen):
    pygame.draw.rect(screen, self.color, self.rect, border_radius=5)
    if self.frozen:
        pygame.draw.rect(screen, (120,200,255), self.rect.inflate(8,8), 3, border_radius=7)
    # eyes
    for ex in [self.rect.x+4, self.rect.x+14]:
        pygame.draw.circle(screen, (255,255,255), (ex, self.rect.y+8), 4)
        pygame.draw.circle(screen, (0,0,0), (ex+1, self.rect.y+8), 2)

```

game/game_engine.py

```

import pygame
from game.maze import generate_maze, CELL
from game.entities import Player, Enemy

COLS, ROWS = 13, 11
WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 80
FPS = 60
SPEED_UP_MS = 15000 # enemies speed up every 15 seconds
SPEED_STEP = 2
MIN_MOVE_INTERVAL = 5
FREEZE_FRAMES = 300 # 5 seconds at 60 FPS
PELLET_CELL = (ROWS//2, COLS//4)

```

```
class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Chase")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.big_font = pygame.font.SysFont("monospace", 38, bold=True)
        self.reset()

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
        self.enemies = [Enemy(ROWS-1, COLS-1), Enemy(0, COLS-1), Enemy(ROWS-1, 0)]
        self.exit_rect = pygame.Rect((COLS//2)*CELL+5, (ROWS//2)*CELL+5, CELL-10, CELL-10)
        self.caught = False
        self.won = False
        self.start_ticks = pygame.time.get_ticks()
        self.speed_tier = 1
        pr, pc = PELLET_CELL
        self.pellet_rect = pygame.Rect(pc*CELL+CELL//2-9, pr*CELL+CELL//2-9, 18, 18)
        self.pellet_active = True
        self.freeze_timer = 0

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT: return False
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r: self.reset()
        return True

    def update(self):
        if self.caught or self.won: return
        keys = pygame.key.get_pressed()
```

```

self.player.move(keys, self.walls, ROWS, COLS)
elapsed = pygame.time.get_ticks() - self.start_ticks
target_tier = elapsed // SPEED_UP_MS + 1
while self.speed_tier < target_tier and any(e.move_interval > MIN_MOVE_INTERVAL for e in self.enemies):
    self.speed_tier += 1
    for enemy in self.enemies:
        enemy.move_interval = max(MIN_MOVE_INTERVAL, enemy.move_interval - SPEED_STEP)
if self.pellet_active and self.player.rect.colliderect(self.pellet_rect):
    self.pellet_active = False
    self.freeze_timer = FREEZE_FRAMES
    for enemy in self.enemies:
        enemy.frozen = True
for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)
    if self.player.rect.colliderect(enemy.rect):
        self.caught = True
if self.freeze_timer > 0:
    self.freeze_timer -= 1
    if self.freeze_timer == 0:
        for enemy in self.enemies:
            enemy.frozen = False
if self.player.rect.colliderect(self.exit_rect):
    self.won = True

def draw(self):
    self.screen.fill((230, 220, 210))
    wc=(50,40,60)
    for r in range(ROWS):
        for c in range(COLS):
            x,y=c*CELL,r*CELL
            w=self.walls[r][c]
            if w[0]: pygame.draw.line(self.screen,wc,(x,y),(x+CELL,y),3)
            if w[1]: pygame.draw.line(self.screen,wc,(x,y+CELL),(x+CELL,y+CELL),3)

```

```

        if w[2]: pygame.draw.line(self.screen,wc,(x+CELL,y),(x+CELL,y+CELL),3)
        if w[3]: pygame.draw.line(self.screen,wc,(x,y),(x,y+CELL),3)
pygame.draw.rect(self.screen,(80,200,80),self.exit_rect,border_radius=4)
lbl=self.font.render("EXIT",True,(20,80,20))
self.screen.blit(lbl,(self.exit_rect.x+2,self.exit_rect.y+6))
if self.pellet_active:
    pygame.draw.circle(self.screen,(255,220,0),self.pellet_rect.center,9)
    pygame.draw.circle(self.screen,(160,120,0),self.pellet_rect.center,9,2)
self.player.draw(self.screen)
for enemy in self.enemies:
    enemy.draw(self.screen)
hud=pygame.Rect(0,ROWS*CELL,WIDTH,80)
pygame.draw.rect(self.screen,(30,30,50),hud)
info=self.font.render("Reach EXIT before the enemy catches you! R=Restart",True,(200,200,200))
self.screen.blit(info,(8,ROWS*CELL+14))
maxed = all(e.move_interval <= MIN_MOVE_INTERVAL for e in self.enemies)
tier=self.font.render(f"Enemy Speed: Tier {self.speed_tier}"+(" (MAX)" if maxed else ""),True,(255,200,80))
self.screen.blit(tier,(8,ROWS*CELL+46))
if self.freeze_timer > 0:
    frz=self.font.render(f"FROZEN: {(self.freeze_timer+59)//60}s",True,(120,200,255))
    self.screen.blit(frz,(WIDTH-frz.get_width()-8,ROWS*CELL+46))
if self.caught:
    self._overlay("CAUGHT!", (220,60,60))
if self.won:
    self._overlay("ESCAPED!", (80,220,80))
pygame.display.flip()

def _overlay(self, text, color):
    surf=pygame.Surface((WIDTH,ROWS*CELL),pygame.SRCALPHA)
    surf.fill((0,0,0,140))
    self.screen.blit(surf,(0,0))
    msg=self.big_font.render(text,True,color)
    sub=self.font.render("Press R to Restart",True,(200,200,200))

```

```

self.screen.blit(msg, (WIDTH//2-msg.get_width()//2, ROWS*CELL//2-30))
self.screen.blit(sub, (WIDTH//2-sub.get_width()//2, ROWS*CELL//2+20))

def run(self):
    running=True
    while running:
        running=self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
    pygame.quit()

```

Summary of changes

- `entities.py`: `Enemy` gets a `frozen` flag. `update()` returns immediately while it's set, so a frozen enemy neither moves nor chases. `draw()` adds a light-blue ice outline around a frozen enemy.
- `game_engine.py`:
 - The pellet is a yellow circle in a fixed open cell, `(row 5, col 3)`. It doesn't overlap the start, the enemy corners or the EXIT, and every cell is reachable in the generated maze.
 - Touching it removes the pellet, sets `frozen` on all 3 enemies and starts a 300-frame countdown. When it hits 0, all enemies unfreeze.
 - The HUD shows `FROZEN: Ns` in blue at the right of the speed line while the freeze is active.
 - `reset()` restores the pellet and clears the freeze.
- **Task 2 progression:** the speed tier keeps running on the wall clock during a freeze. Enemies resume at whatever interval they've reached.

Headless tests passed, and I checked the rendered frames by eye:

- I walked the player to the pellet with simulated key presses.
- The enemies stay frozen for exactly 300 frames with zero movement, and all 3 resume chasing on frame 301.
- The pellet works only once, and R clears the pellet and freeze state.
- Task 1 catching and Task 2 tiers still work.

I couldn't test real-time feel or real keyboard input, so please run the checklist.

One design choice: frozen enemies can still catch you if you touch them, because the README only says they stop moving. If you want frozen enemies to be harmless, that's a one-line change in `update()`. I left it out to stay within scope.

Manual test checklist

1. Launch the game. You should see a yellow circular pellet in the left-middle area, in an open cell away from the EXIT and the corners.
2. Wait a few seconds and confirm the enemies are chasing normally, then walk to the pellet. It should vanish immediately.
3. On pickup:
 - All 3 enemies stop, with a light-blue outline around each.
 - The HUD shows `FROZEN: 5s` at the right and counts down to 1s.
 - Watch the counter as well as the clock, since the freeze lasts about 5 seconds.
4. When the countdown ends, the outlines and HUD text disappear and all 3 enemies resume chasing.
5. Collect the pellet after 15 seconds or more. The speed tier in the HUD should still advance during the freeze, and the enemies should resume at the faster speed.
6. Press R during a freeze. The pellet should reappear, the freeze should be gone, and the tier should return to 1.
7. Confirm you can't pick up the pellet a second time in the same run, and that tasks 1 and 2 still behave as before.

Task 4 — Score by Distance / Survival Score

Tasks 1, 2, and 3 have already been implemented and tested successfully.

Using the current project code, implement only Task 4: Score by Distance / Survival Score exactly as specified in README.md.

Requirements:

- Increase the score by 1 every frame while the player is alive and the game is active.
- Display survival time in the HUD as `Survived: Xs`, where $X = \text{score} // 60$.
- When the player is caught, show the final score/survival result on the game-over overlay.
- Restarting with R must reset the score to 0.

Important:

- Preserve the existing implementations of:
 - Task 1: Multiple Enemies
 - Task 2: Speed Up Over Time
 - Task 3: Power Pellet / Freeze
- Preserve all original game functionality.
- Change existing code only where strictly necessary for Task 4.
- Do not rewrite, refactor, optimize, or fix unrelated code.
- Do not add unrelated features, files, or dependencies.
- Keep changes minimal and consistent with the existing code style.

After implementing Task 4, also do a final review of the project against README.md and confirm whether all 4 tasks are implemented. Do not make additional changes during this review unless they are strictly required by one of the four README tasks.

Return:

1. The complete updated contents only for files that need modification, clearly labeled with their file paths.
2. Put the code in directly copy-pastable code blocks.
3. Do not use diff/patch format or partial snippets. Give the complete contents of each modified file.
4. Give a very short summary of the Task 4 changes.
5. Give a final manual test checklist covering Task 4 and confirming Tasks 1–4 work together.

Assistant

30/09/2026, 16:38:20

► Wording the game-over overlay to display the final score.

Task 4 only needs `game/game_engine.py`. The HUD bar's two lines are already used, so the survival timer will need a third line. First I'll apply the edit.

► Apply Task 4 edits and diff vs Task 3 version `bash_tool`

That's the shell tripping on a leftover stray command of mine, not a problem with the edit. I'll check what actually ran and diff again cleanly.

► Re-diff Task 4 changes `bash_tool`

► Diff Task 3 vs Task 4 engine using bash `bash_tool`

The diff has exactly the intended Task 4 changes. Now I'll test the score behaviour, then run a combined test of all four tasks.

► Headless test of Task 4 score behaviour `bash_tool`

Now the visuals: the HUD after a restart and the CAUGHT overlay with the result.

► Check the CAUGHT overlay showing final score `view`

► Debugging the frozen enemy state and score overlay display.

► Reviewing README requirements against the actual game code

► Compare current project against the original ZIP `bash_tool`

Only the two intended files differ from the original. Now the requirement-by-requirement check against README.md and a real-time run of the actual game loop, with no mocked clock.

► Verify all four README tasks against the code `bash_tool`

► Print final `game_engine.py` `bash_tool`

Task 4 changed only `game/game_engine.py`. `entities.py` stays as you already have it from Task 3.

`game/game_engine.py`

```
import pygame
from game.maze import generate_maze, CELL
from game.entities import Player, Enemy

COLS, ROWS = 13, 11
WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 110
FPS = 60
SPEED_UP_MS = 15000 # enemies speed up every 15 seconds
SPEED_STEP = 2
```

Copy

```
MIN_MOVE_INTERVAL = 5
FREEZE_FRAMES = 300 # 5 seconds at 60 FPS
PELLET_CELL = (ROWS//2, COLS//4)

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Chase")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.big_font = pygame.font.SysFont("monospace", 38, bold=True)
        self.reset()

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
        self.enemies = [Enemy(ROWS-1, COLS-1), Enemy(0, COLS-1), Enemy(ROWS-1, 0)]
        self.exit_rect = pygame.Rect((COLS//2)*CELL+5, (ROWS//2)*CELL+5, CELL-10, CELL-10)
        self.caught = False
        self.won = False
        self.start_ticks = pygame.time.get_ticks()
        self.speed_tier = 1
        pr, pc = PELLET_CELL
        self.pellet_rect = pygame.Rect(pc*CELL+CELL//2-9, pr*CELL+CELL//2-9, 18, 18)
        self.pellet_active = True
        self.freeze_timer = 0
        self.score = 0

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT: return False
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r: self.reset()
```

```

        return True

def update(self):
    if self.caught or self.won: return
    self.score += 1
    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)
    elapsed = pygame.time.get_ticks() - self.start_ticks
    target_tier = elapsed // SPEED_UP_MS + 1
    while self.speed_tier < target_tier and any(e.move_interval > MIN_MOVE_INTERVAL for e in self.enemies):
        self.speed_tier += 1
        for enemy in self.enemies:
            enemy.move_interval = max(MIN_MOVE_INTERVAL, enemy.move_interval - SPEED_STEP)
    if self.pellet_active and self.player.rect.colliderect(self.pellet_rect):
        self.pellet_active = False
        self.freeze_timer = FREEZE_FRAMES
        for enemy in self.enemies:
            enemy.frozen = True
    for enemy in self.enemies:
        enemy.update(self.walls, self.player, ROWS, COLS)
        if self.player.rect.colliderect(enemy.rect):
            self.caught = True
    if self.freeze_timer > 0:
        self.freeze_timer -= 1
        if self.freeze_timer == 0:
            for enemy in self.enemies:
                enemy.frozen = False
    if self.player.rect.colliderect(self.exit_rect):
        self.won = True

def draw(self):
    self.screen.fill((230, 220, 210))
    wc=(50,40,60)

```

```

for r in range(ROWS):
    for c in range(COLS):
        x,y=c*CELL,r*CELL
        w=self.walls[r][c]
        if w[0]: pygame.draw.line(self.screen,wc,(x,y),(x+CELL,y),3)
        if w[1]: pygame.draw.line(self.screen,wc,(x,y+CELL),(x+CELL,y+CELL),3)
        if w[2]: pygame.draw.line(self.screen,wc,(x+CELL,y),(x+CELL,y+CELL),3)
        if w[3]: pygame.draw.line(self.screen,wc,(x,y),(x,y+CELL),3)
pygame.draw.rect(self.screen,(80,200,80),self.exit_rect,border_radius=4)
lbl=self.font.render("EXIT",True,(20,80,20))
self.screen.blit(lbl,(self.exit_rect.x+2,self.exit_rect.y+6))
if self.pellet_active:
    pygame.draw.circle(self.screen,(255,220,0),self.pellet_rect.center,9)
    pygame.draw.circle(self.screen,(160,120,0),self.pellet_rect.center,9,2)
self.player.draw(self.screen)
for enemy in self.enemies:
    enemy.draw(self.screen)
hud=pygame.Rect(0,ROWS*CELL,WIDTH,110)
pygame.draw.rect(self.screen,(30,30,50),hud)
info=self.font.render("Reach EXIT before the enemy catches you! R=Restart",True,(200,200,200))
self.screen.blit(info,(8,ROWS*CELL+14))
maxed = all(e.move_interval <= MIN_MOVE_INTERVAL for e in self.enemies)
tier=self.font.render(f"Enemy Speed: Tier {self.speed_tier}"+(" (MAX)" if maxed else ""),True,(255,200,80))
self.screen.blit(tier,(8,ROWS*CELL+46))
if self.freeze_timer > 0:
    frz=self.font.render(f"FROZEN: {(self.freeze_timer+59)//60}s",True,(120,200,255))
    self.screen.blit(frz,(WIDTH-frz.get_width()-8,ROWS*CELL+46))
surv=self.font.render(f"Survived: {self.score//60}s",True,(200,200,200))
self.screen.blit(surv,(8,ROWS*CELL+78))
if self.caught:
    self._overlay("CAUGHT!", (220,60,60))
if self.won:
    self._overlay("ESCAPED!", (80,220,80))

```

```

pygame.display.flip()

def _overlay(self, text, color):
    surf=pygame.Surface((WIDTH,ROWS*CELL),pygame.SRCALPHA)
    surf.fill((0,0,0,140))
    self.screen.blit(surf,(0,0))
    msg=self.big_font.render(text,True,color)
    sub=self.font.render("Press R to Restart",True,(200,200,200))
    self.screen.blit(msg,(WIDTH//2-msg.get_width()//2,ROWS*CELL//2-30))
    self.screen.blit(sub,(WIDTH//2-sub.get_width()//2,ROWS*CELL//2+20))
    res=self.font.render(f"Final Score: {self.score} (Survived: {self.score//60}s)",True,(255,255,255))
    self.screen.blit(res,(WIDTH//2-res.get_width()//2,ROWS*CELL//2+52))

def run(self):
    running=True
    while running:
        running=self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
    pygame.quit()

```

Summary of Task 4 changes

- Score: `self.score` is set to 0 in `reset()` and incremented by 1 per frame in `update()`, after the caught/won early return. It stops when the game ends, keeps counting during a freeze, and R resets it.
- HUD: a third line shows `Survived: Xs`, where X is `score // 60`. I made the HUD bar 30px taller (80 to 110) to fit it, since the first two lines were already full.

- Overlay: the game-over overlay now adds `Final Score: N (Survived: Xs)` under "Press R to Restart". It also appears on the ESCAPED overlay, because both use the same `_overlay()` function, and the game ends either way.

Final review against README.md

All 4 tasks are implemented and pass every check. I compared the current project against your original ZIP: only `game/game_engine.py` and `game/entities.py` differ, and `main.py`, `maze.py`, `requirements.txt` and the README are byte-identical. Line endings are still CRLF.

Task	README requirement	Status
1	3 enemies in different corners in <code>self.enemies</code> , each calling <code>bfs()</code> on its own	✓
2	<code>get_ticks()</code> , every 15s reduce <code>move_interval</code> by 2, minimum 5, tier shown in HUD	✓
3	Yellow pellet, 300-frame freeze, <code>enemy.frozen</code> , <code>update()</code> does nothing while frozen, indicator on enemy	✓
4	<code>score += 1</code> per frame, <code>Survived: Xs</code> , final score on overlay	✓

I also ran the real `run()` loop unmocked for 120 frames. It took about 1.96s and the score reached 120, so it runs at 60 FPS with no errors.

Things I noticed but left alone, because none belongs to the four tasks:

- The original `Player._valid()` only checks the maze boundary, not the wall lines, so the player can walk through walls. You may want to ask your instructor whether that's intended.
- The HUD text still says "the enemy" in the singular.
- In my test font the first HUD line is cut off at the right edge ("R=Restart" is clipped). On your machine the width depends on your font.
- The README checklist item "Code reviewed with LLM (include chat link)" is something only you can do. Don't forget the chat link.

Final manual test checklist

Task 4

1. Launch the game. The HUD's third line should read `Survived: 0s`, and it should tick up once per second.
2. Get caught. The overlay should show `Final Score: N (Survived: Xs)`. For example, about 10 seconds alive gives roughly `Final Score: 600 (Survived: 10s)`. The number should stay fixed while the overlay is showing.
3. Press R. The score and HUD should go back to `Survived: 0s`.
4. Reach the EXIT. The score stops and shows on the ESCAPED overlay too.

Tasks 1 to 4 together

5. At the start there are 3 red enemies in three corners, the yellow pellet is at left-middle, the tier reads 1, and the timer reads 0s.
6. After about 15 seconds the tier rises to 2 and all enemies speed up, while the survival timer keeps counting.
7. Grab the pellet. All 3 enemies freeze with blue outlines, `FROZEN: 5s` counts down, the pellet vanishes, and the survival timer and tier keep advancing. After 5 seconds all 3 enemies resume chasing.
8. Press R in the middle of a freeze. You get a new maze, the pellet is back, the enemies are unfrozen and back at their corners, the tier is 1 and the score is 0.
9. Let it run about 2 minutes 15 seconds to see `Tier 9 (MAX)`, with enemies never faster than the floor.

To make the testing quicker, you can temporarily set `SPEED_UP_MS = 2000`. Set it back to `15000` before you submit.